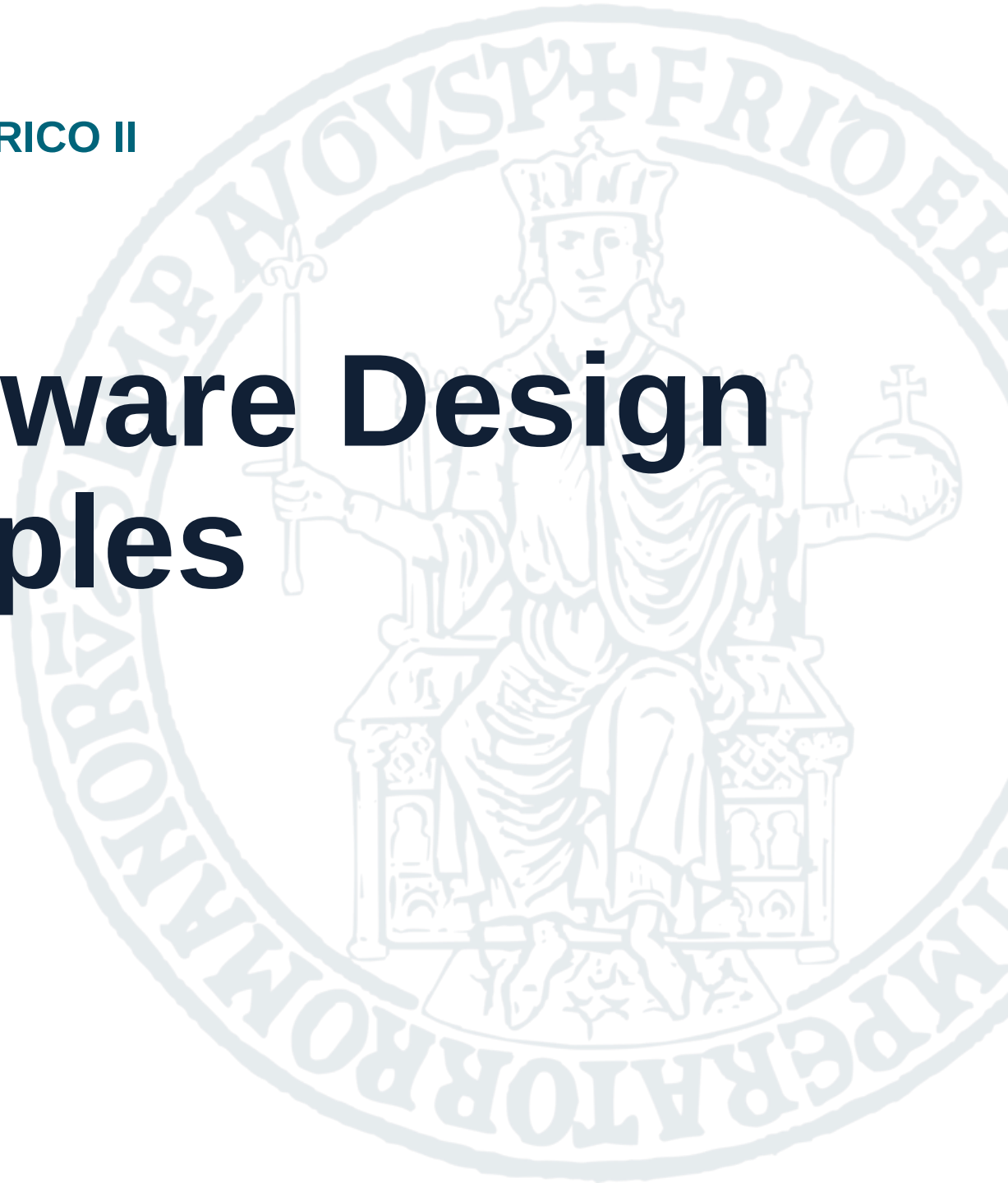
UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

SOFTWARE ENGINEERING

Improving Software Design – SOLID Principles

Prof. Sergio Di Martino




SOLID
Software Development is not a Jenga game

Software “Smells”

- The system is rigid
  - Hard to change because everything has to change at once
  - Shotgun Surgery
- The system is fragile
  - Changes cause the system to break in the strangest of places.
- The system is immobile
  - That is, not reusable.
- The system is opaque
  - Hard to understand.
- The system is needlessly complex
- The system contains needless repetition.
- Have you ever seen software with these problems?

SOLID Design Principles

- Introduced by Robert C. Martin between 2000 and 2003
  - *Design Principles and Design Patterns, Object Mentor* 1.34 (2000): 597
  - *Agile Principles, Patterns, and Practices in C#*

- **S**ingle Responsibility Principle
- **O**pen/Closed Principle
- **L**iskov Substitution Principle
- **I**nterface Segregation Principle
- **D**ependency Inversion Principle

Single Responsibility Principle



Single Responsibility Principle

- A class should have only ONE Responsibility
- A class should have only **ONE** reason to change
- A responsibility is “a reason for change.”
- The responsibilities of a class are axes of change.
  - If it has two responsibilities, they are coupled in the design, and so have to change together.

- If a class has more that one responsibility, it may become fragile when any of the requirements change

Single Responsibility Principle – Example

Report
GetData() FormatReport() Print()

```
namespace Dimecasts.SOLID
{
    class Report
    {
        private IList<ReportDataElement> GetData()
        {
            Console.WriteLine("\nGetting Data...");
            return new List<ReportDataElement>() { new Repo
        }

        private void FormatReport()
        {
            Console.WriteLine("\nFormatting Report...");
        }

        public void Print()
        {
            GetData();
            FormatReport();
            Console.WriteLine("\nPrinting Report...");
        }
    }
}
```

Single Responsibility Principle – Example V2

```
public class DataAccess
{
    public IList<ReportDataElement> GetData()
    {
        Console.WriteLine("\nGetting Data...");
        return new List<ReportDataElement>() { new ReportDataElement()
    }
}
```

```
public class ReportFormatter
{
    public void FormatReport()
    {
        Console.WriteLine("\nFormatting Report...");
    }
}
```

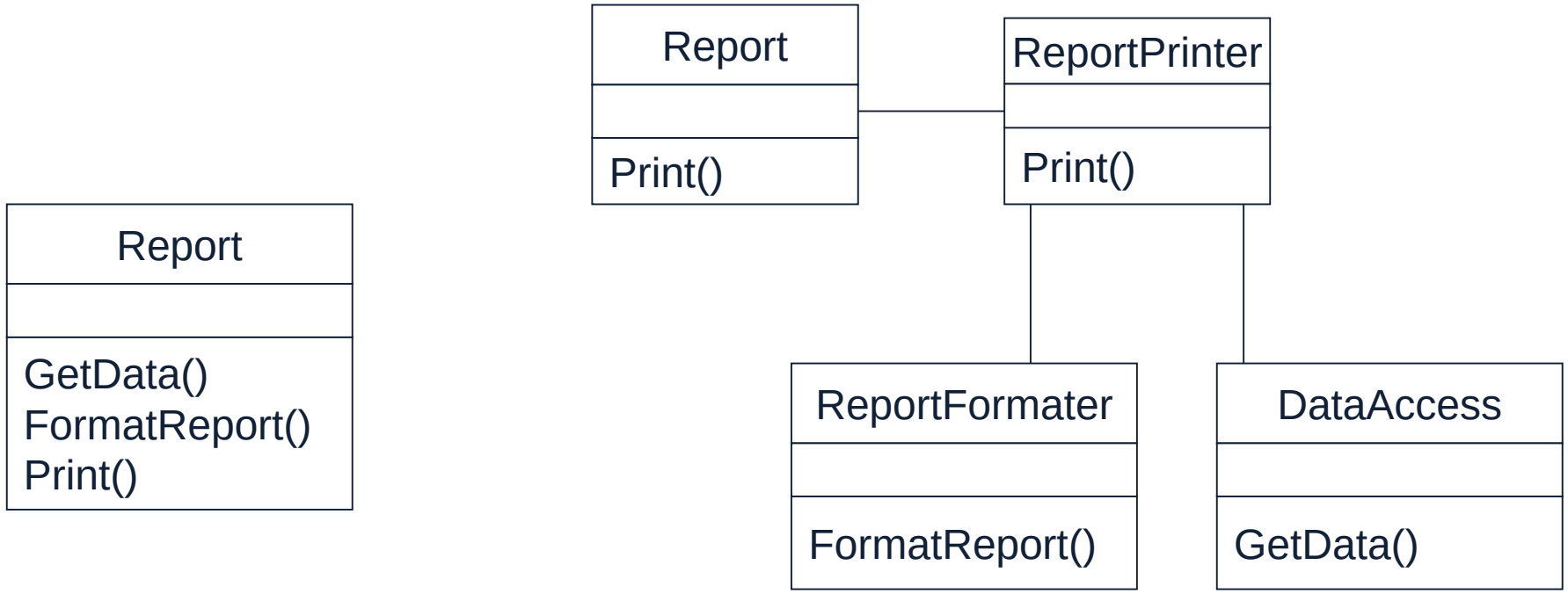
Single Responsibility Principle – Example V2

```
public class ReportPrinter
{
    public void Print()
    {
        DataAccess dataAccess = new DataAccess();
        ReportFormatter reportFormatter = new ReportFormatter();
        dataAccess.GetData();
        reportFormatter.FormatReport();
        Console.WriteLine("\nPrinting Report...");
    }
}
```

```
class Report
{

    public void Print()
    {
        ReportPrinter reportPrinter = new ReportPrinter();
        reportPrinter.Print();
    }
}
```

Single Responsibility Principle – Example v2



Benefits of SRP

- Reuse - If all your classes follow the SRP, you are more likely to reuse some of them
- Clarity - Your code is cleaner as your classes don’t do unexpected things
- Naming - As all your classes have a single responsibility, choosing a good name is easy
- Readability - Due to the clarity, better names and shorter files the readability is improved greatly.

Open/Closed Principle



Open/Closed Principle

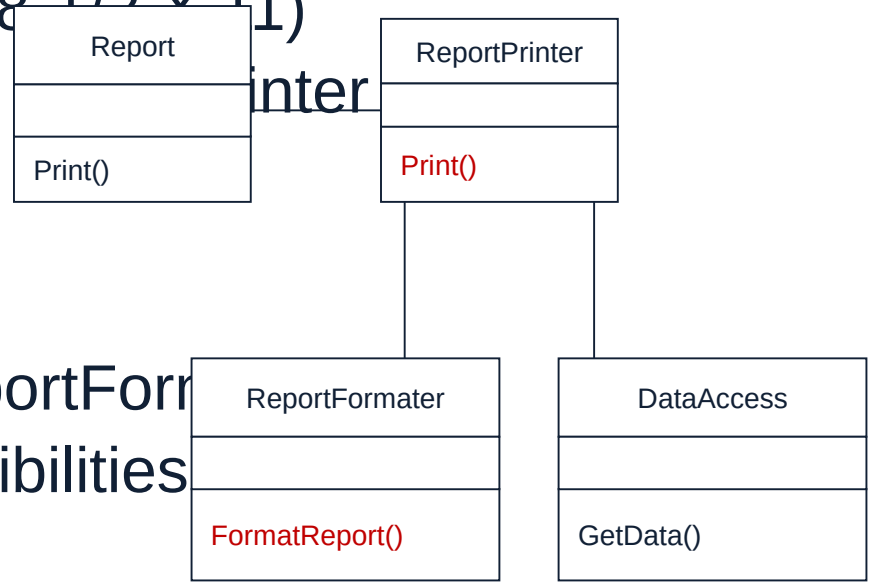
- Software entities (module, classes, functions, etc.) should be open for extension but closed for modification
- Classes should be written so that they can be extended without requiring modification.
  - extend the behaviour of a system (in response to a requested change)
  - add new code
  - But not modifying the existing code

Open/Closed Principle

- Once completed, the implementation of a class should only be modified to correct errors; new or changed features would require that a different class be created
- Mechanism: Abstraction and subtype polymorphism are the keys to the OCP
  - Introducing Abstract Base Classes
  - Implementing common Interfaces
  - Deriving from common interfaces or classes

OCP – Example

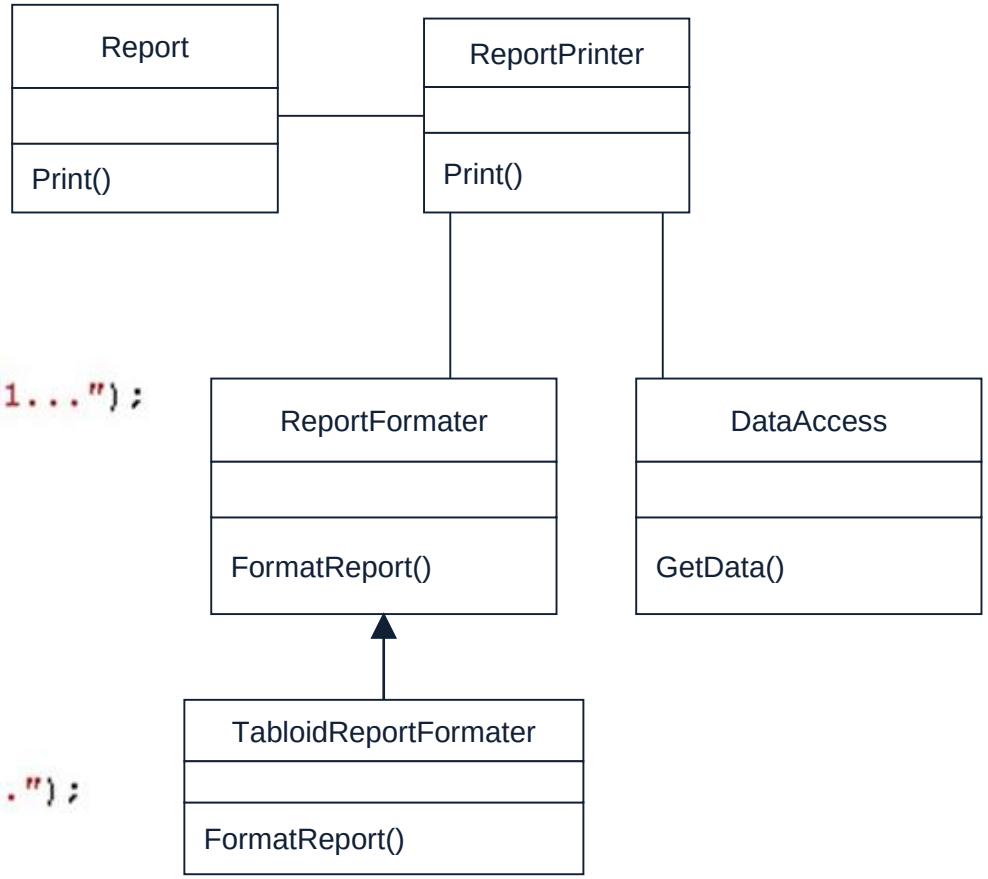
- We get a request to change the print requirement
  - Format the report into Tabloid (instead of 8 1/2 X 11)
  - Print the report to a dot-matrix (instead of [illegible]inter
- Solution 1:
  - Modify the code in ReportPrinter and ReportForr[illegible]
  - Add some "if" to handle the different possibilities
- Solution 2:
  - Add specialized subclasses



OCP – Example ct.

```
public class ReportFormatter
{
    public virtual void FormatReport()
    {
        Console.WriteLine("\nFormatting Report for 8-1/2 X 11...");
    }
}

public class TabloidReportFormatter : ReportFormatter
{
    public override void FormatReport()
    {
        Console.WriteLine("\nFormatting Report for 11 X 17...");
        //base.FormatReport();
    }
}
```

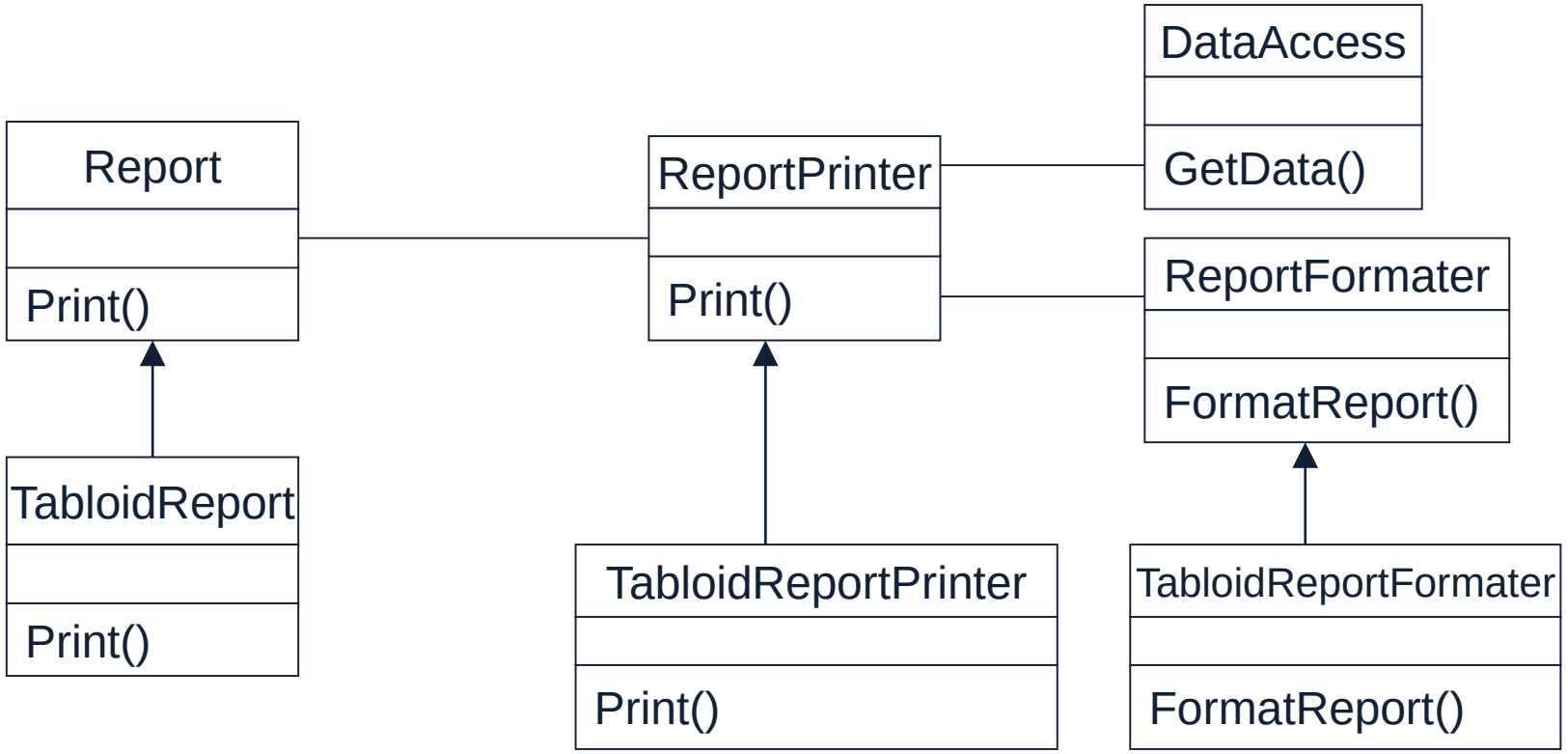


OCP – Example cnt.

```
public class ReportPrinter
{
    public void Print()
    {
        DataAccess dataAccess = new DataAccess();
        ReportFormatter reportFormatter = new ReportFormatter();
        dataAccess.GetData();
        reportFormatter.FormatReport();
        Console.WriteLine("\nPrinting Report to laser printer...");
    }
}

public class TabloidReportPrinter : ReportPrinter
{
    public override void Print()
    {
        DataAccess dataAccess = new DataAccess();
        ReportFormatter reportFormatter = new TabloidReportFormatter();
        dataAccess.GetData();
        reportFormatter.FormatReport();
        Console.WriteLine("\nPrinting Report to dot-matrix printer...");
        //base.Print();
    }
}
```

OCP – Final Class Diagram



OCP – Example cnt.

```
class Report
{

    public virtual void Print()
    {
        ReportPrinter reportPrinter = new ReportPrinter();
        reportPrinter.Print();
    }
}


public class TabloidReport : Report
{
    public override void Print()
    {
        ReportPrinter reportPrinter = new TabloidReportPrinter();
        reportPrinter.Print();
        //base.Print();
    }
}
```

Note *the extension of the system behaviour without making modification to existing code*

Benefits of OCP

- design is more stable
  - existing (working) code does not change
  - changes are made by adding, not modifying, code
  - changes are isolated and do not cascade throughout the code
- code is potentially easier to test
- change is easier to manage
  - code is easier to replace
  - design is extensible
- code is potentially reusable
- Higher abstraction

Liskov Substitution Principle



Liskov Substitution Principle (LSP)

- Subclasses should be substitutable for their base class
- LSP states that if a program module is using a Base class, then the reference to the Base class can be replaced with a Derived class without affecting the functionality of the program module.
- “A derived class should have some kind of specialized behaviour (it should provide the same services as the superclass, only some, at least, are provided differently.)”
- The contract of the base class must be honoured by the derived class.

Liskov Substitution Principle (LSP)

- In basic Object-Oriented principles, “Inheritance” is usually described as an "is a" relationship.
- If a “Developer” is a “SoftwareProfessional”, then, the “Developer” class should inherit the “SoftwareProfessional” class.
- Such “Is a” relationship is very important in class designs, but it is easy to get carried away and end up in wrong design with bad inheritance.
- The “Liskov's Substitution Principle” is a way of ensuring that inheritance is used correctly.

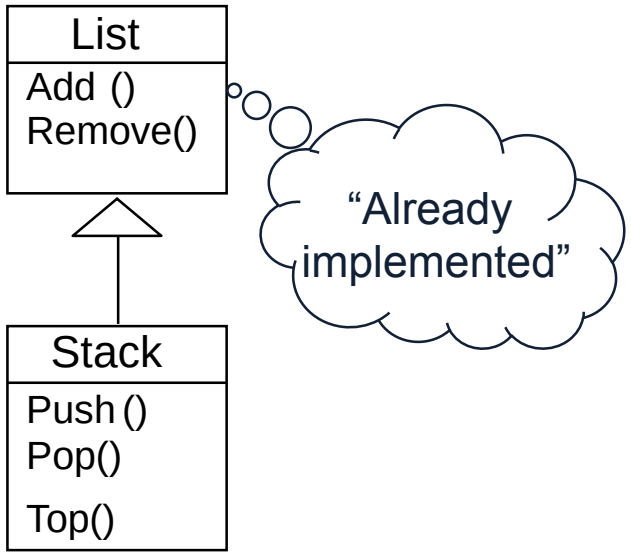
Violazione LSP

```
public class Rectangle {
    private double height;
    private double width;
    public void setHeight(double
h) { height = h; }
    public void setWidht(double
w) { width = w; }
    ...
}
```

```
public class Square extends
Rectangle {
    public void setHeight(double h)
{
        super.setHeight(h);
        super.setWidth(w);
    }
    public void setWidth(double h)
{
        super.setHeight(h);
        super.setWidth(w);
    }
}
```

Wrong Implementation Inheritance

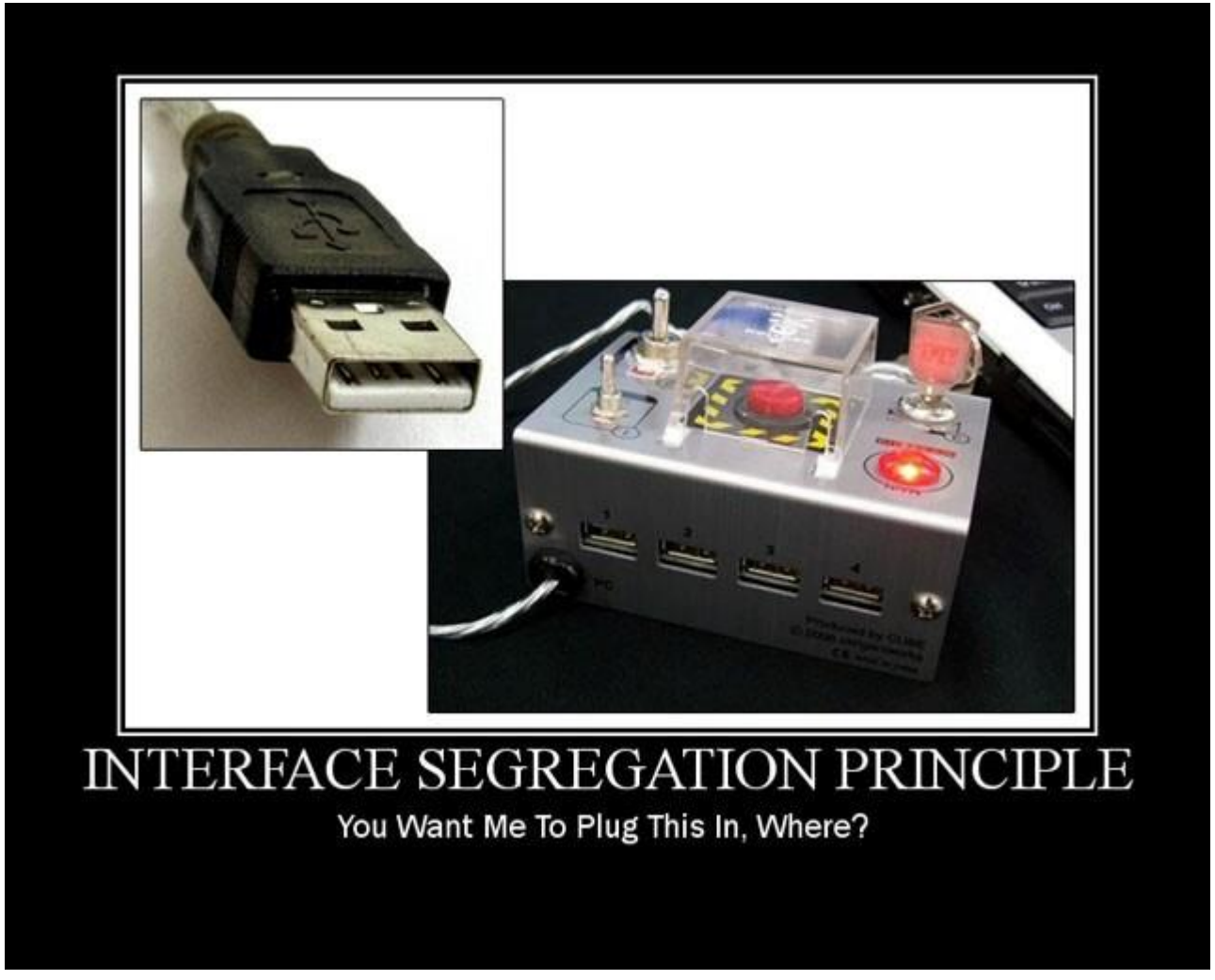
- A very similar class is already implemented that does almost the same as the desired class implementation.
- Example: I have a List class, I need a Stack class. How about subclassing the Stack class from the List class and providing three methods, Push() and Pop(), Top()?



LSP – Benefits

- If LSP is not maintained:
  - class hierarchies would be a mess and if subclass instance was passed as parameter to methods method, strange behavior might occur.
  - unit tests for the Base classes would never succeed for the subclass.
- If this principle is violated, then it will cause the Open-Closed Principle to be violated also.

Interface Segregation Principle



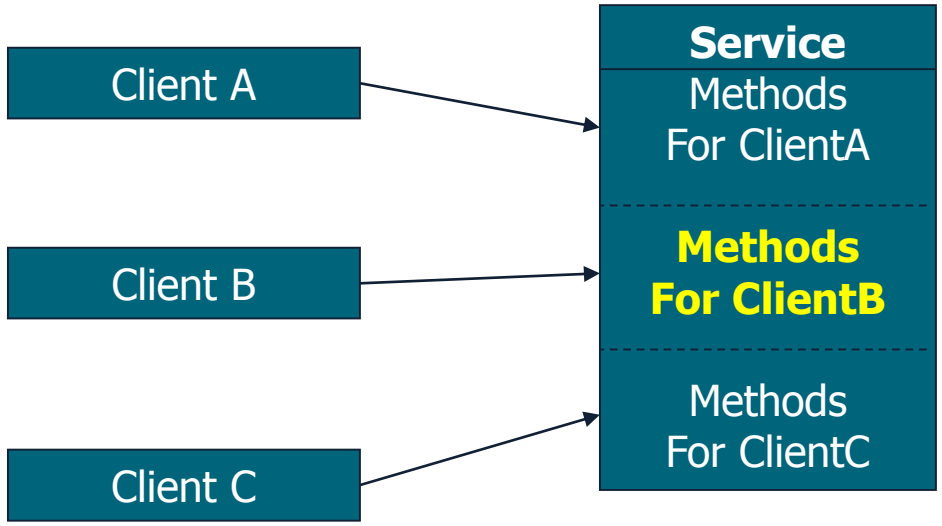
Interface Segregation Principle (ISP)

- This principle was first defined by Robert C. Martin as: “**Clients should not be forced to depend upon interfaces that they do not use**“.
- Many client-specific interfaces are better than one general purpose interface
  - Interfaces should be thin and not fat
  - E.g. if you have an interface with 20 methods, then not all implementers of that interface might not implement all of those methods
    - Bring large and fat interfaces into smaller ones which classify the related methods

Interface Segregation Principle (ISP)

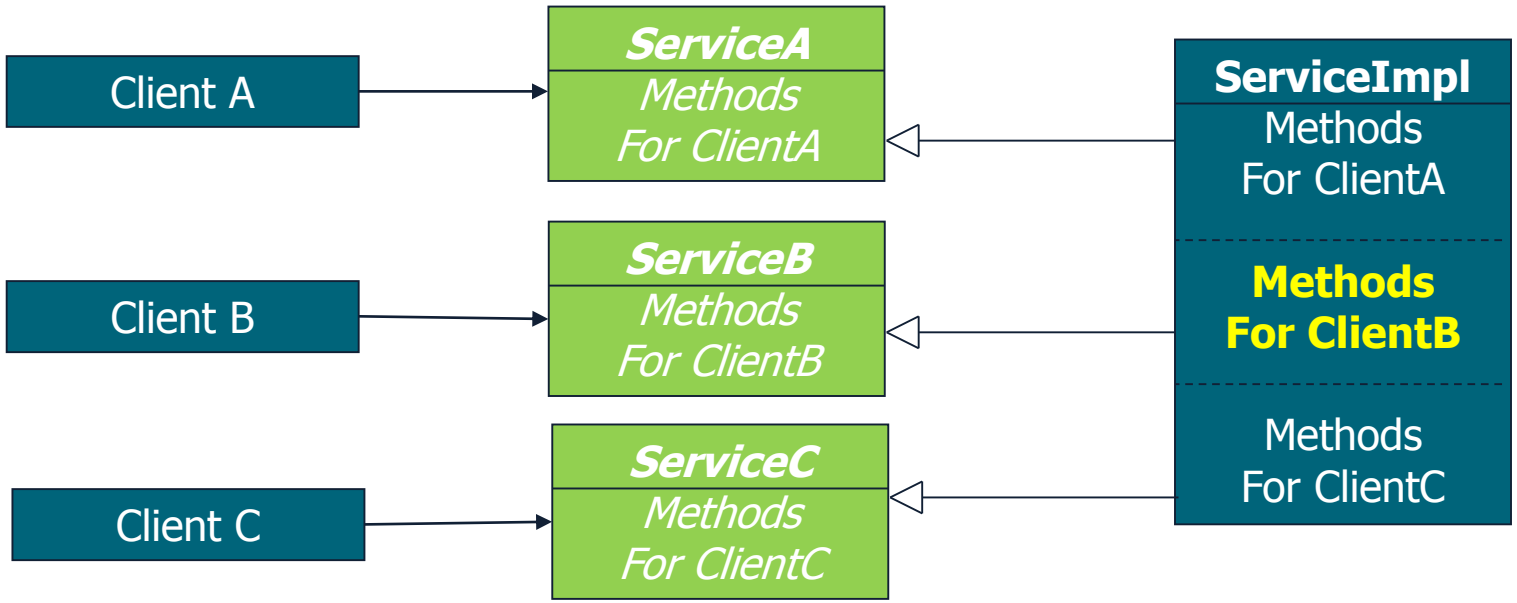
- If you have a class with several different uses, create separate (narrow) interfaces for each use.
- The purpose is to make clients use as small and as coherent an interface as possible.
- Fat interfaces lead to inadvertent couplings and accidental dependencies between classes.

ISP – A Bad Design



- The problem is that any change in Service introduced by one client (e.g., Client A) causes Service and the other clients (Client A and Client C) to be recompiled.

ISP – A Good Design



- Changes introduced by one client (e.g., Client A) may require its designated interface (ServiceA) and the service implementation to change.
- However, other clients (Client A & Client C) and their Interfaces (ServiceB & ServiceC) are not required to change or recompile.

ISP – Another Example

```
public interface Vehicle {
    public void drive();
    public void stop();
    public void refuel();
    public void openDoors();
}
```

```
public class MotorBike implements Vehicle {

    // Can be implemented
    public void drive() {...}
    public void stop() {...}
    public void refuel() {...}

    // Can not be implemented
    public void openDoors() {...}
}
```

ISP – Yet Another Example

```
public interface IPrinterTasks
{
    void Print(string PrintContent);
    void Scan(string ScanContent);
    void Fax(string FaxContent);
    void PrintDuplex(string PrintDuplexContent);
}
```

Forcing a class to provide the body of an interface method means violating the Interface Segregation Principle

```
class HPLaserJetPrinter : IPrinterTasks
{
    public void Print(string PrintContent)
    {
        Console.WriteLine("Print Done");
    }

    public void Scan(string ScanContent)
    {
        Console.WriteLine("Scan content");
    }

    public void Fax(string FaxContent)
    {
        Console.WriteLine("Fax content");
    }

    public void PrintDuplex(string PrintDuplexContent)
    {
        Console.WriteLine("Print Duplex content");
    }
}
```

```
class LiquidInkjetPrinter : IPrinterTasks
{
    public void Print(string PrintContent)
    {
        Console.WriteLine("Print Done");
    }

    public void Scan(string ScanContent)
    {
        Console.WriteLine("Scan content");
    }

    public void Fax(string FaxContent)
    {
        throw new NotImplementedException();
    }

    public void PrintDuplex(string PrintDuplexContent)
    {
        throw new NotImplementedException();
    }
}
```

ISP – Another Example V2

```
public interface IPrinterTasks
{
    void Print(string PrintContent);
    void Scan(string ScanContent);
}
interface IFaxTasks
{
    void Fax(string content);
}
interface IPrintDuplexTasks
{
    void PrintDuplex(string content);
}
```

Splitting a big interface into smaller ones

```
public class HPLaserJetPrinter : IPrinterTasks, IFaxTasks,
                                 IPrintDuplexTasks
{
    public void Print(string PrintContent)...

    public void Scan(string ScanContent)...

    public void Fax(string FaxContent)...

    public void PrintDuplex(string PrintDuplexContent)...
}
```

Class implementing all the three interfaces means this class wants all the four services

```
class LiquidInkjetPrinter : IPrinterTasks
{
    public void Print(string PrintContent)...

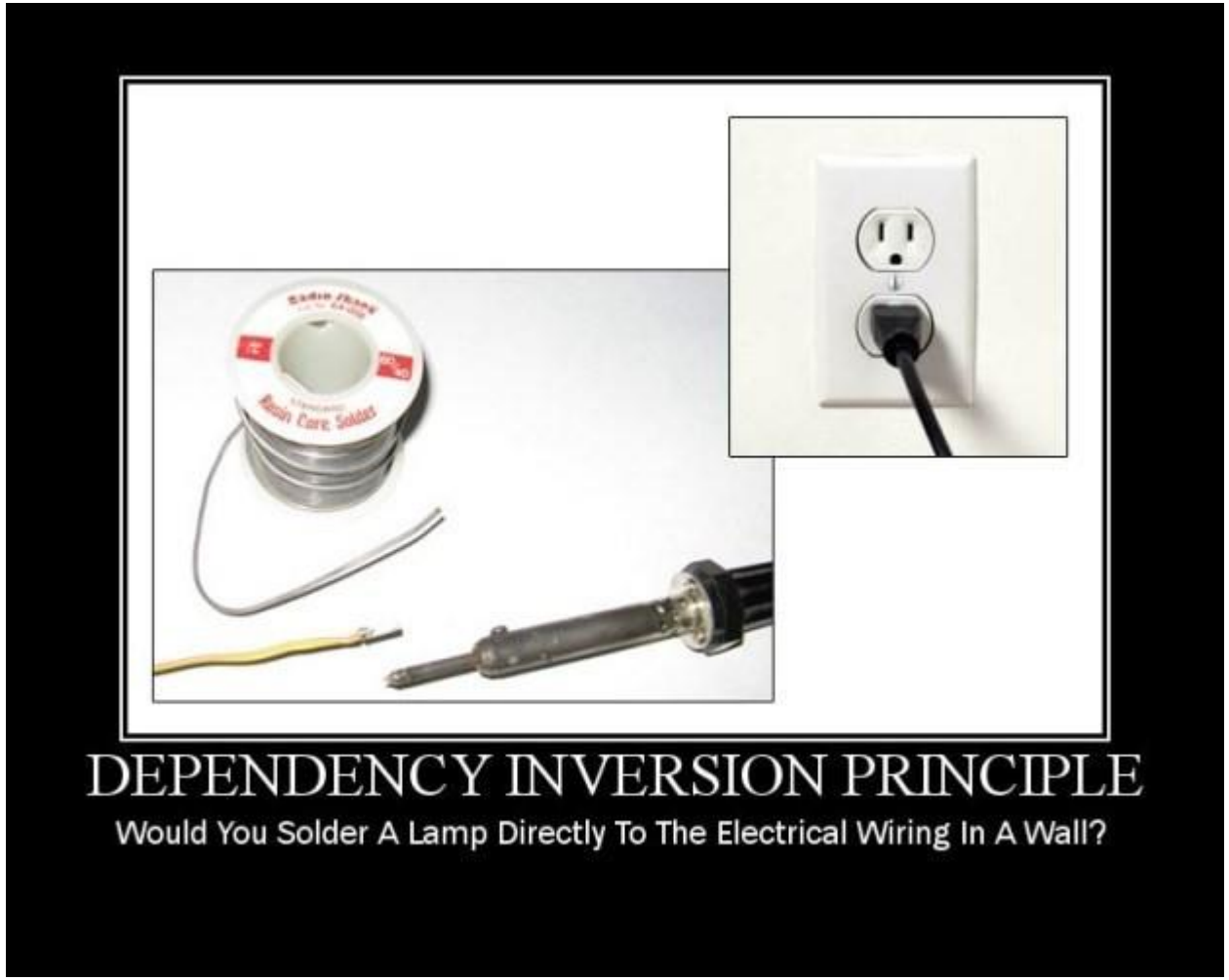
    public void Scan(string ScanContent)...
}
```

This class wants only Print and Scan service so implementing only the IPrinterTasks interface

ISP – Benefits

- Cohesion is increased
  - clients can demand cohesive interfaces
- Design is more stable
  - changes are isolated and do not cascade throughout the code
- Supports the Liskov Substitution Principle

Dependency Inversion Principle



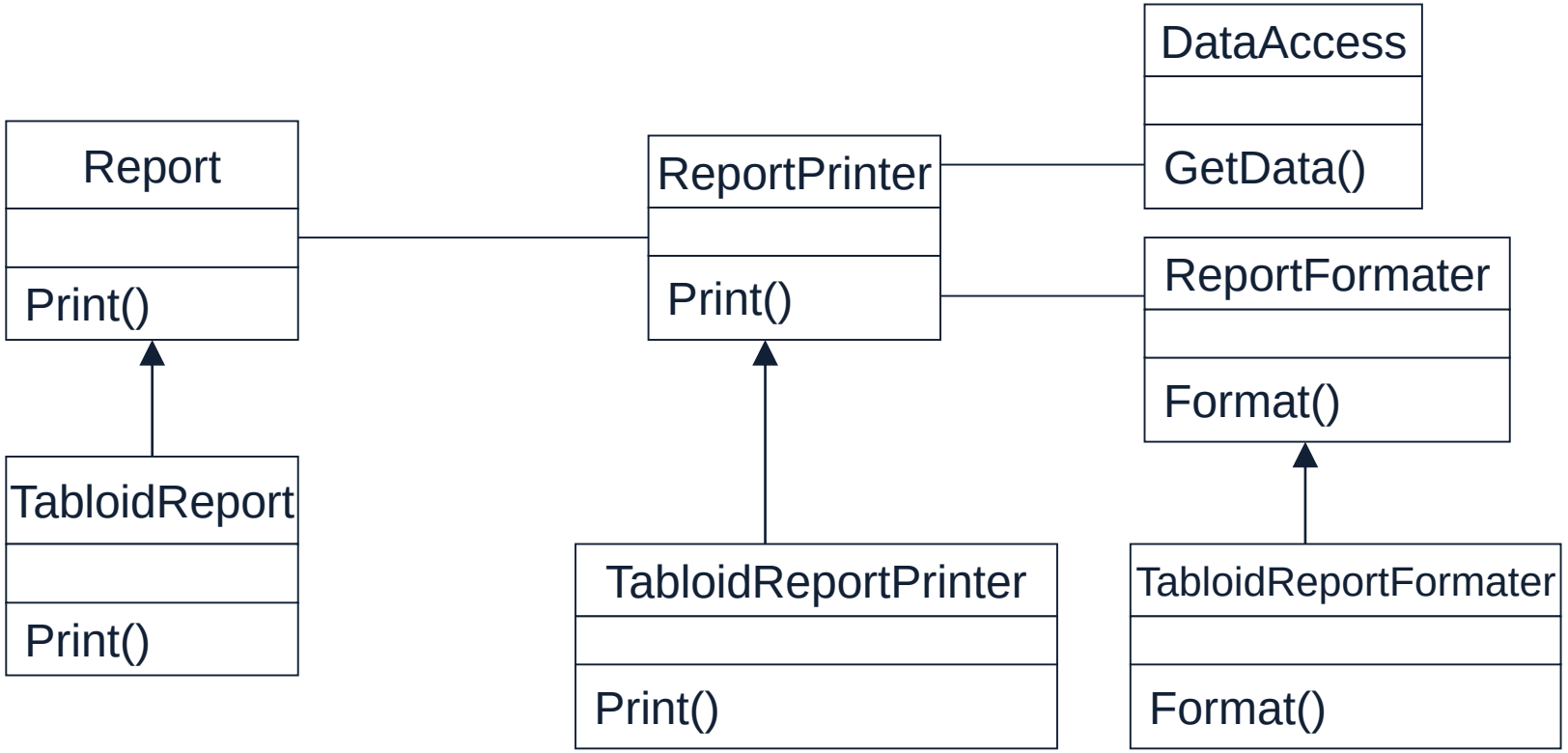
Dependency Inversion Principle (DIP)

- High level modules should not depend on low level modules
  - both should depend upon abstractions
    - Dependency upon lower-level components limits the reuse opportunities of the higher-level components
- Abstractions should not depend on details
- Change the relation between concrete classes to relationships among abstractions
  - Abstraction mechanism: Abstract base classes or Interfaces

Dependency Inversion Principle (DIP)

- The goal of the dependency inversion principle is to
  - decouple high-level components from low-level components
  - such that reuse with different low-level component implementations becomes possible
- High level classes should contain the “business logic”, low level classes should contain the details of the (current) implementation.
- Access instances using interfaces or abstract classes to avoid dependency inversion.

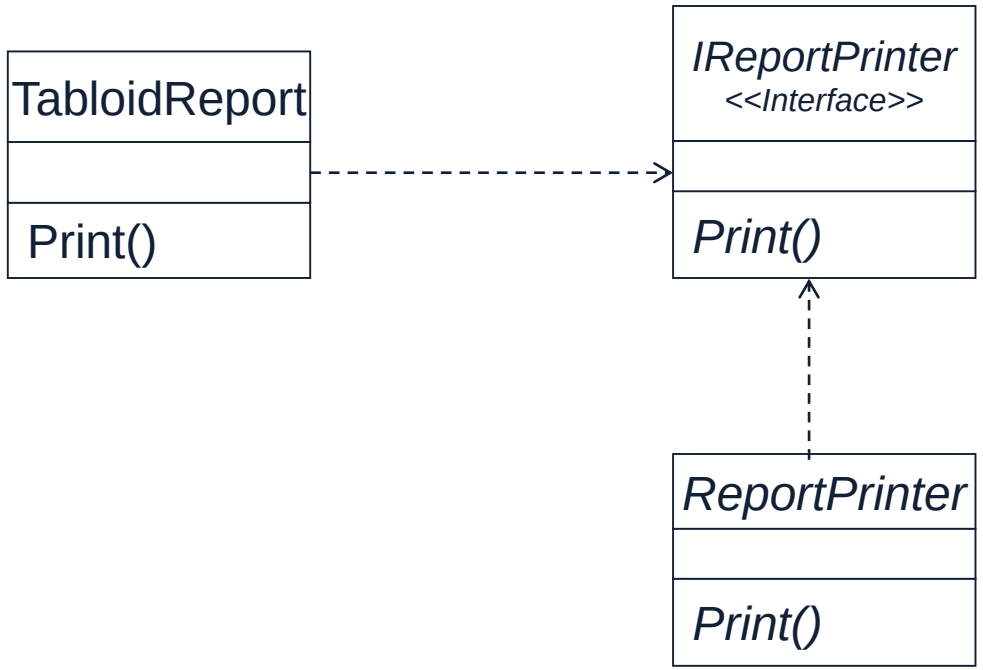
DIP – Example


Report
Print()
TabloidReport
Print()
ReportPrinter
Print()
TabloidReportPrinter
Print()
DataAccess
GetData()
ReportFormater
Format()
TabloidReportFormater
Format()

DIP

```
internal class TabloidReport : Report
{
    public override void Print()
    {
        ReportPrinter reportPrinter = new TabloidReportPrinter();
        reportPrinter.Print();
        //base.Print();
    }
}
```





DIP

```
internal class TabloidReport : Report
{
    public override void Print()
    {
        ReportPrinter reportPrinter = new TabloidReportPrinter();
        reportPrinter.Print();
        //base.Print();
    }
}
```

```
internal interface IReportPrinter
{
    void Print();
}
abstract class ReportPrinter : IReportPrinter
{
    public abstract void Print();
}
```

```
internal class TabloidReport : Report
{
    public override void Print()
    {
        IReportPrinter reportPrinter = new TabloidReportPrinter();
        reportPrinter.Print();
        //base.Print();
    }
}
```

DIP – Benefits

- Enables design by contract
- change is easier to manage
  - code is easier to replace
  - design is extensible
- code is potentially easier to test

Sources

- Example slides are partly adopted from
  - http://dimecasts.net/Casts/ByTag/SOLID%20Principle